DISTRIBUTED ALGORITHMS • SHORTEST PATH • ROUTING PROTOCOLS

Distributed Bellman-Ford: Distance Vectors & Negative Cycles

Synthesis based on Arpit Bhayani's Distributed Systems Masterclass • Bellman (1958) & Ford (1956)

ALGORITHMIC PREMISE & DISTANCE VECTOR MECHANICS

In arbitrary weighted networks (where edge costs denote latency, tariffs, or bandwidth bottlenecks), shortest paths cannot be computed by simple hop count. The **Distributed Bellman-Ford algorithm** generalizes shortest path finding to arbitrary directed graphs with arbitrary real-valued weights. By executing localized **Distance Vector Relaxations** across synchronous rounds, nodes converge to optimal shortest paths in at most $|V| - 1$ rounds and detect **Negative Weight Cycles** at round $|V|$.

1. MOTIVATION: WEIGHTED SHORTEST PATH VS. HOP COUNT

A 1-hop high-latency link (600 ms satellite) is significantly slower than a 3-hop low-latency route (15 ms fiber). While Dijkstra's algorithm requires a centralized priority queue, Bellman-Ford operates via purely localized edge relaxations between adjacent routers.

2. FORMAL MODEL & SYSTEM STATE

Graph Topography

- **Directed Weighted Graph:** $G = (V, E)$ with edge weights $w(u, v) \in \mathbb{R}$.
- **Designated Source s:** Computes single-source shortest paths (SSSP).
- **Synchronous Rounds:** Lock-step rounds $1 \dots |V|$.

Node Variables (Node u)

- $d(u)$: Shortest distance estimate ($d(s)=0, d(u)=\infty$).
- $ ext{parent}(u)$: Predecessor hop on the shortest path.

3. THE DISTANCE VECTOR RELAXATION STEP

For any directed edge $(u, v) \in E$, when node u advertises its distance estimate $d(u)$, neighbor v relaxes its local distance vector:

$$ ext{If } d(u) + w(u, v) < d(v) \implies d(v) \gets d(u) + w(u, v), \quad ext{parent}(v) \gets u$$

In each round, nodes broadcast updated estimates to outgoing neighbors, iteratively propagating shortest-path improvements outward.

4. FORMAL PROOF OF CONVERGENCE IN |V| - 1 ROUNDS

Assume $G = (V, E)$ has no negative-weight cycles reachable from source s:

Lemma 1: Shortest Paths are Simple

In the absence of negative cycles, every shortest path contains no repeated vertices (no cycles). Since there are $|V|$ total vertices, any simple path has at most $|V| - 1$ edges.

Lemma 2: Induction on Path Edge Count

Let $P = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from $v_0 = s$ to $v_k = v$ with $k \le |V| - 1$ edges:

- **Base Case ($k = 0$):** At round 0, $d(s) = 0 = \delta(s, s)$. Trivially optimal.
- **Inductive Step:** Assume node v_{k-1} converged to $\delta(s, v_{k-1})$ by round $k-1$. In round k, node v_{k-1} advertises its distance to v_k. During relaxation, $d(v_k) \le \delta(s, v_{k-1}) + w(v_{k-1}, v_k) = \delta(s, v_k)$.

Because the longest simple path has $|V| - 1$ edges, **all vertices reach optimal shortest distances within $|V| - 1$ rounds**.

5. NEGATIVE CYCLE DETECTION: THE |V|-TH ROUND TEST

If a graph contains a cycle whose weight sum is strictly negative ($\sum_{(u, v) \in C} w(u, v) < 0$), paths can achieve distance $-\infty$ by traversing the cycle indefinitely.

Negative Cycle Invariant: Run relaxation for round $|V|$. If any node's distance estimate strictly decreases during round $|V|$ ($d(v)_{|V|} < d(v)_{|V|-1}$), then G contains a negative-weight cycle reachable from s!

6. COMPLEXITY ANALYSIS & BOUNDS

Time Complexity: $\mathcal{O}(|V|)$ Rounds

The algorithm terminates in at most $|V| - 1$ rounds for convergence, plus 1 round for negative cycle detection. Total time: $\mathcal{O}(|V|)$ synchronous rounds.

Message Complexity: $\mathcal{O}(|V| \cdot |E|)$

In each round, every edge carries at most 1 distance vector update. Total messages across all rounds: bounded by $\mathcal{O}(|V| \cdot |E|)$ messages.

7. DISTANCE VECTOR PROTOCOLS & INTERNET ROUTING DYNAMICS

Protocol Dilemma	Mechanism & Failure Mode	Production Routing Mitigation
Fast Good News	New low-latency fiber links propagate across the network in k rounds.	Immediate distance vector relaxation.
Count-to-Infinity	When a link breaks, mutual neighbors iteratively update each other with stale paths, slowly incrementing to ∞.	**Split Horizon with Poison Reverse:** Never advertise a route back to the gateway from which it was learned (set cost to ∞).
Transient Loops	Asynchronous delays cause out-of-order distance vector processing.	**Path Vector Routing (BGP):** Explicitly include Autonomous System (AS) path list; drop routes containing own AS.

8. SYSTEMS ENGINEERING TAKEAWAYS

1. **Local Updates Lead to Global Convergence:** Bellman-Ford proves that routers do not require complete map views of the Internet; exchanging vectors exclusively with adjacent physical neighbors guarantees optimal routing tables.
2. **Financial Arbitrage Modeling:** Bellman-Ford's negative cycle detection is the standard engine for forex arbitrage: setting edge weight $w(u, v) = -\ln(ext{Exchange Rate}(u, v))$ allows negative cycles to uncover profitable circular trade loops.
3. **Memory vs. Convergence Trade-off:** Bellman-Ford uses $O(ext{degree})$ memory per router but suffers the count-to-infinity penalty under network partitions. Link-state protocols (OSPF) use $O(|V| + |E|)$ memory to achieve instantaneous SPF tree recalculation.